Ramverk

Namn: Michaela Ståhlgren
Kurs: Avancerad webbutveckling med HTML och CSS (AWH201)
Datum: 2026-03-29
GitHub repo: https://github.com/michastahl/xexinterior.git

Innehåll

1. Inledning ... 3
2. Presentation av olika ramverk ... 3
2.1 Bootstrap ... 3
2.2 Foundation ... 4
2.3 Bulma ... 4
2.4 Tailwind CSS ... 4
2.5 Mitt val av ramverk: Tailwind CSS ... 5
3. Layout/Mall/Template ... 5
4. Brytpunkter ... 6
5. Egen CSS ... 6
6. Optimering av media ... 7
7. Kompabilitet ... 7
8. Validering, Prestanda- och kvalitetsanalys med Lighthouse ... 8
9. Utvecklingsprocess ... 9
10. Summering ... 10
11. Referenser ... 11
12. Bilagor ... 13
12.1 Bilaga 1- Produkt-grid ... 13
Bild 1. Exempel på produkt-grid som visar hur produkter presenteras i ett responsivt rutnät. ... 13
12.2 Bilaga 2- Egen CSS ... 14
Bild 1. Utdrag ur projektets egna CSS-fil. ... 14
12.3 Bilaga 3- Responsiv bildhantering med picture-element ... 15
Bild 1. Kodexempel som visar hur olika hero-bilder laddas beroende på skärmstorlek. ... 15
12.4 Bilaga 4- Validering av HTML ... 16
Bild 1. Resultat från W3C HTML Validator som visar identifierade varningar i koden. ... 16
12.5 Bilaga 5- Lighthouse-analys ... 17
Bild 1. Översikt av Lighthouse-resultat (mobiltest) ... 17
Bild 2. Identifierade problem inom SEO ... 17
Bild 3. Identifierade problem inom Best Practices ... 17

1. Inledning

Syftet med uppgiften var att utveckla en flersidig webbplats med hjälp av ett CSS-ramverk och därigenom arbeta mer effektivt än vid traditionell CSS-utveckling. Uppgiften fokuserade på att använda ramverkets egna klasser och komponenter i så hög grad som möjligt, i stället för att skriva egen CSS.

Jag har valt att arbeta med ramverket Tailwind och att utgå från min tidigare webbplatsidé från WUH-kursen och vidareutveckla den till en större webbplats med flera undersidor. Fokus låg på att skapa en sammanhållen design där samma visuella uttryck återkommer på alla sidor, samtidigt som varje sida har ett eget innehåll och en egen funktion. I koden syns att projektet bygger på Tailwind CSS tillsammans med Vite, och att den egna CSS-koden har hållits mycket begränsad. Det gjorde uppgiften särskilt relevant, eftersom den i hög grad följer kursens krav om att låta ramverket bära största delen av layout- och designarbetet.

Ett annat mål med arbetet var att förstå hur val av ramverk påverkar både arbetsprocess och slutresultat. När man arbetar utan ramverk har man mycket stor frihet, men man behöver också själv lösa sådant som grid-system, spacing, typografisk skala och responsivitet. Med ett ramverk får man i stället ett mer styrt arbetssätt. Det kan upplevas som begränsande, men det ger samtidigt en tydlig struktur och gör det lättare att arbeta konsekvent mellan flera sidor. Arbetet med denna uppgift har därför inte bara handlat om att skapa en fungerande webbplats, utan också om att reflektera över hur verktygsval, kodkvalitet, tillgänglighet och prestanda samverkar i modern frontendutveckling.

2. Presentation av olika ramverk

CSS-ramverk används för att effektivisera utvecklingen av webbgränssnitt genom att erbjuda färdiga lösningar för layout, typografi, komponenter och responsivitet. Användning av ramverk minskar behovet av att skriva CSS från grunden, vilket leder till snabbare utveckling och mer konsekventa användargränssnitt (Bootstrap, u.å.). Många morderna ramverk är dessutom konstruerade enligt principen mobile-first, vilket ligger i linje med rekommendationer kring responsiv webbdesign (MDN Web Docs, 2025). Samtidigt innebär ramverk alltid ett visst mått av standardisering. Ju mer man använder ett ramverks färdiga struktur, desto större är risken att webbplatsen får ett uttryck som liknar andra sidor byggda med samma verktyg. Därför blir valet av ramverk både en teknisk och en designmässig fråga. Det handlar inte bara om vad som är möjligt, utan också om hur väl verktyget passar projektets mål, arbetsflöde och önskad detaljkontroll.

2.1 Bootstrap

Bootstrap är ett av de mest etablerade frontend-ramverken och beskrivs i den officiella dokumentationen som mobile-first. Ramverket erbjuder ett tydligt gridsystem, färdiga komponenter som navbarer, cards, knappar och formulär samt en relativt låg tröskel för att komma igång. Det gör Bootstrap särskilt användbart i projekt där snabb utveckling, god

dokumentation och förutsägbar funktion prioriteras. Fördelen är att mycket fungerar direkt utan omfattande anpassning. Nackdelen är att Bootstrap lätt får ett igenkännbart standardutseende om man inte aktivt förändrar färger, typografi och komponenternas uttryck. I ett projekt där den visuella identiteten redan är tydligt definierad kan detta innebära att man behöver skriva mer kompletterande CSS än man först tror (Bootstrap, u.å.).

2.2 Foundation

Foundation beskriver sig som ett avancerat responsivt front-end-ramverk med fokus på flexibilitet och anpassning. Jämfört med Bootstrap framstår Foundation som mer inriktat på semantik och på att ge utvecklaren större kontroll över strukturen. Det kan vara en styrka i större projekt där skalbarhet och anpassningsbarhet är viktiga. Samtidigt är tröskeln ofta högre, både eftersom dokumentationen kan upplevas som mindre direkt för nybörjare och eftersom man behöver förstå fler delar av ramverket för att utnyttja det effektivt. Foundation lämpar sig därför bättre när man vill bygga mer avancerade lösningar än när målet främst är att snabbt ta fram en färdig studentwebbplats. (Foundation, u.å.).

2.3 Bulma

Bulma är ett CSS-ramverk som bygger på Flexbox och erbjuder färdiga klasser och komponenter och kräver ingen JavaScript-funktionalitet från ramverket självt. I dokumentationen beskrivs Bulma som ett modernt och responsivt CSS-ramverk. En fördel med Bulma är att det är lätt att läsa och använda, eftersom klassnamnen ofta är ganska tydliga. Begränsningen ligger i att Bulma ofta styr uttrycket mer än utility-first-ramverk gör. För ett projekt som redan har en etablerad layoutidé kan det därför bli svårare att återskapa den exakta designen utan att komplettera med egen CSS (Bulma, u.å.).

2.4 Tailwind CSS

Tailwind CSS skiljer sig från de mer traditionella ramverken genom att vara utility-first. I stället för att utgå från färdiga komponenter arbetar man med små klasser direkt i HTML, till exempel för padding, margin, grid, flex, färger, typografi och responsiva brytpunkter. Tailwind beskriver själv denna metod som att man bygger designen direkt i markup (Tailwind CSS, u.å.-e). Fördelen är att det ger mycket hög kontroll över det visuella resultatet utan att man behöver skriva stora mängder egen CSS. Samtidigt blir HTML-koden ofta mer omfattande, och arbetssättet kräver att man vänjer sig vid att tänka i kombinationer av klasser snarare än i separata stilregler. För utvecklare som vill översätta en egen design till ett ramverk är Tailwind därför ofta ett starkt alternativ.

2.5 Mitt val av ramverk: Tailwind CSS

Jag valde Tailwind CSS eftersom jag utgick från en tidigare designidé och ville behålla ett eget visuellt uttryck utan att skriva omfattande egen CSS. Just där passade Tailwind bra, eftersom jag kunde översätta tidigare designbeslut till utility-klasser för exempelvis maxbredder, flex, grid, spacing, typografi, färger och avrundade hörn. I projektets kod syns detta tydligt genom att största delen av layouten ligger direkt i klasslistorna i HTML, medan den egna CSS-filen i princip bara innehåller grundläggande typografiska regler för body, rubriker, stycken och formulärelement.

Jag valde bort Bootstrap, det hade sannolikt blivit enklare att snabbt få fram en funktionell navigation och färdiga komponenter, men svårare att anpassa uttrycket till min tidigare layout utan att skriva mer egen CSS. Jag valde bort Bulma av liknande skäl. Ramverket är lätt att läsa, men det är mer komponentorienterat än Tailwind och hade därför sannolikt krävt fler kompromisser kring struktur och proportioner. Foundation övervägdes mindre i praktiken, främst eftersom det kändes tyngre än vad projektet krävde. För just denna uppgift, där målet var att kombinera ett eget uttryck med så lite egen CSS som möjligt, gav Tailwind bäst balans mellan frihet och struktur.

Valet av Tailwind påverkade också mitt sätt att tänka kring utveckling. I stället för att fråga mig vilka CSS-regler jag behövde skriva började jag tänka i färdiga byggstenar: hur många kolumner en sektion skulle ha, vilka avstånd som behövdes, hur text skulle skalas mellan mobil och desktop och hur samma komponentmönster kunde återanvändas på flera sidor. Det gjorde arbetet mer systematiskt och ligger nära det arbetssätt som ofta används i modern frontendutveckling (Tailwind CSS, u.å.-d).

Genom att arbeta praktiskt med flera ramverk har jag fått en djupare förståelse för hur tekniska val påverkar både arbetsflöde och slutresultat. Det har också blivit tydligt att valet av ramverk inte enbart handlar om funktionalitet, utan även om hur väl det passar den egna arbetsmetoden.

3. Layout/Mall/Template

Webbplatsen bygger på en sammanhållen grundlayout där alla sidor delar centrala designelement: wrapper med maxbredd, mörk header, hero-bild, återkommande footer och samma typografiska grund. Det skapar igenkänning och gör att användaren förstår att sidorna hör ihop. Samtidigt har varje sida ett eget innehåll och en egen funktion. Startsidan fungerar som en introduktion och innehåller utvalda produkter, inspirationsbilder, nyhetsbanner och formulär för nyhetsbrev. Produktsidan för dekoration fördjupar sig mer i ett specifikt innehåll genom produkt-gridar och inspirationssektioner. Informationssidorna använder i högre grad kombinationer av bild och text, kortliknande informationsblock och gridlösningar för exempelvis butiker och kontaktuppgifter.
Det här valet av layout är motiverat utifrån innehållets karaktär. Produkter behöver visas på ett överskådligt sätt, vilket gör grid-lösningar lämpliga (Tailwind CSS, u.å.-b).

Informationssidor kräver i stället mer luft och tydlig läsordning, vilket gör tvåkolumnslayouter med bild och text mer användbara. I projektet används båda dessa principer konsekvent. På så sätt blir webbplatsen varierad utan att kännas spretig.

Navigationen är responsiv och förändras beroende på skärmstorlek. På mindre skärmar används en hamburgermeny som aktiveras med JavaScript, eftersom MDN beskriver responsiv navigation och tillgänglighet (MDN Web Docs, 2025). En styrka här är att navigationsmönstret återkommer på alla sidor, vilket ger god orienterbarhet. En förbättringsmöjlighet är däremot att desktopversionens dropdown bygger på hover, vilket fungerar visuellt men kan göras mer tillgängligt med tydligare tangentbordsstöd och synkroniserade aria-attribut.

4. Brytpunkter

I projektet används Tailwinds mobile-first-princip tillsammans med brytpunkterna *md*, *lg* och *xl*. Detta innebär att standardlayouten gäller för mobil, och större skärmar får sedan tillägg genom prefixed utility-klasser. Det är ett arbetssätt som överensstämmer väl med Tailwinds dokumentation, där oprefixerade klasser gäller för alla skärmstorlekar och prefixed klasser träder in från respektive brytpunkt och uppåt (Tailwind CSS, u.å.-d).

I praktiken innebär detta att innehållet ofta visas i en kolumn på mobil, vilket förbättrar läsbarheten och minskar risken för trånga layouter. Vid md delas flera sektioner upp i två kolumner, till exempel kombinationer av bild och text eller produktlistor. På större skärmar används xl för bredare grids med tre eller fyra kolumner. Ett tydligt exempel är startsidans produktsektion, som går från en kolumn till två och vidare till fyra kolumner (se Bilaga 1, Bild 1.). Det finns också exempel på lg på vissa informationssidor, där kontakt- eller informationsblock får två kolumner först på större skärmar.
Att först utgå från mobil bidrar till bättre läsbarhet och minskar risken för komplexa och svårnavigerade layouter, vilket är en central rekommendation inom responsiv webbdesign (MDN Web Docs, 2025).

5. Egen CSS

Ett viktigt krav i uppgiften var att använda så lite egen CSS som möjligt, och här ligger projektet nära intentionen i uppgiften. Den egna CSS-filen är mycket kort och innehåller huvudsakligen typografiska grundinställningar: font-family och textfärg på body, line-height för rubriknivåer och stycken samt en regel som gör att input, textarea och button ärver webbplatsens font. (se Bilaga 2, Bild 1.). All övrig layout, spacing, färgsättning, rundade hörn, grid-struktur, fokusmarkeringar och responsiva anpassningar sker i huvudsak direkt i HTML med hjälp av Tailwinds utility-klasser.

Detta är en styrka ur både kursens och kodkvalitetens perspektiv. Projektet visar att ramverket faktiskt används som huvudverktyg, inte bara som ett lager ovanpå traditionell CSS. Samtidigt finns en tydlig personlig prägel. Färgskalan bygger till stor del på mörka neutrala toner kombinerade med ljus bakgrund och en guldfärgad accent i fokusmarkeringar.

Typografin hålls konsekvent genom användningen av Google Font Roboto. Resultatet blir därför inte ett anonymt standardutseende, trots att väldigt lite egen CSS skrivits.

En relevant teknisk detalj är att projektet inte använder någon separat reset-fil som Normalize.css. Det är rimligt, eftersom Tailwind inkluderar Preflight, ett lager av basstilar som jämnar ut webbläsares standardskillnader och tar bort exempelvis standardmarginaler på rubriker och stycken (Tailwind CSS, u.å.-c). Det innebär att det redan finns en normaliserad grund att bygga vidare på.

6. Optimering av media

Projektet innehåller framför allt bilder, vilket gör mediaoptimering central för både prestanda och användarupplevelse. Här har flera bra tekniker använts. För det första används WebP för nästan alla innehållsbilder. Det är ett modernt bildformat som generellt ger bättre kompression och mindre filstorlekar än traditionella format som JPEG och PNG vid liknande visuell kvalitet (Cloudinary, 2025). I projektet används även picture-elementet för hero- och bannerbilder, vilket gör det möjligt att leverera olika bildfiler beroende på skärmstorlek (MDN Web Docs, 2025). Detta innebär att mindre enheter inte behöver ladda lika stora och tunga bilder som desktopversionen, vilket bidrar till förbättrad prestanda och kortare laddningstid. Implementeringen krävde en förändring i mitt arbetssätt, eftersom jag tidigare främst arbetat enligt en desktop-first-princip. I detta projekt behövde jag i stället planera bildhanteringen utifrån ett mobile-first-perspektiv och därefter successivt anpassa bildstorlekar och layout för större skärmar. Detta gav en ökad förståelse för hur responsiv bildhantering påverkar både teknisk prestanda och användarupplevelse. (se Bilaga 3, Bild 1)

För det tredje används loading="lazy" på många bilder längre ned på sidan. Lazy loading är en etablerad strategi för att skjuta upp hämtningen av resurser som inte behövs direkt när sidan öppnas. På så sätt minskar mängden data som måste laddas initialt, vilket kan förbättra upplevd prestanda, särskilt på mobil eller långsammare nätverk (Pollard, 2024). I projektet är detta särskilt relevant eftersom flera sidor innehåller många produktbilder och inspirationssektioner.

Ytterligare en styrka är att bilderna i koden har width- och height-attribut. Det hjälper webbläsaren att reservera plats i layouten innan bilden är helt nedladdad och minskar därmed risken för layoutförskjutningar (MDN Web Docs, 2025). Sammantaget visar projektet alltså en medveten hantering av bildformat, bildstorlek och laddningsstrategi. En möjlig vidareutveckling hade varit att komplettera de mest centrala bilderna högst upp på sidan med ännu tydligare prioritering, exempelvis genom preload (Pollard & Weiss, 2025) eller genom att analysera vilka bilder som bör laddas tidigast för bästa användarupplevelse.

7. Kompabilitet

Eftersom webbplatsen bygger på moderna frontendtekniker är det viktigt att reflektera över webbläsarstöd. I projektet används bland annat CSS Grid, object-fit, WebP-bilder och lazy loading via loading-attributet. Enligt Can I Use har både CSS Grid och object-fit mycket brett

stöd i moderna webbläsare, men saknar eller har begränsat stöd i äldre miljöer, framför allt Internet Explorer 11 (Can I Use, 2026). WebP stöds också i de flesta moderna webbläsare, men saknas i äldre versioner av Safari och Internet Explorer. Lazy loading via loading-attributet har i dag mycket hög global täckning, men även där finns begränsningar i äldre webbläsare.
För detta projekt är dessa begränsningar acceptabla eftersom webbplatsen tydligt riktar sig till moderna enheter och moderna webbläsare. Det är därför rimligt att prioritera effektivitet och modern standard framför omfattande fallback-lösningar. Samtidigt bör man som utvecklare vara medveten om konsekvenserna. Om webbplatsen hade varit en kommersiell produkt med krav på bred historisk kompatibilitet hade det varit klokt att överväga fallback-bilder i JPEG eller PNG och mer defensiv hantering av layout beroende på målgruppens behov (Can I Use, 2026).

8. Validering, Prestanda- och kvalitetsanalys med Lighthouse

För att säkerställa korrekt kodstruktur validerades samtliga HTML-filer med hjälp av W3C HTML Validator (World Wide Web Consortium, u.å.) Resultatet visade att koden i huvudsak var korrekt och fri från allvarliga syntaxfel. De varningar som förekom gällde främst att vissa section-element saknade rubriker (se Bilaga 4, Bild 1). Detta är i första hand en fråga om semantik och tillgänglighet, eftersom rubriker bidrar till en tydligare dokumentstruktur för både användare och hjälpmedel som skärmläsare. I projektet användes section i vissa fall främst som layout-container snarare än som semantiskt avgränsat innehåll.

Webbplatsens tekniska kvalitet analyserades även med Lighthouse i Chrome DevTools, där tester genomfördes i mobil-läge för att simulera realistiska användningsförhållanden såsom mindre skärmar, lägre processorkraft och långsammare nätverk (Chrome Developers, u.å.) Detta är särskilt relevant eftersom ramverket bygger på en mobile-first-princip.

En översikt av Lighthouse-resultatet visas i Bilaga 5, Bild 1. Resultaten visade hög nivå av tillgänglighet (Accessibility), vilket tyder på en tydlig sidstruktur med korrekt användning av rubriker, alternativtexter för bilder och fungerande formulär. Detta förbättrar användbarheten och gör webbplatsen mer kompatibel med hjälpmedel som skärmläsare.

Under kategorin Best Practices identifierades vissa förbättringsområden, bland annat att några bilder hade relativt låg upplösning i förhållande till sin visningsstorlek (se Bilaga 5, Bild 3). Detta kan påverka den visuella upplevelsen på högupplösta skärmar, men mindre bildfiler bidrar samtidigt till snabbare laddningstider. Resultatet visar därmed den avvägning som ofta krävs mellan prestanda och bildkvalitet i modern webbdesign.

SEO-analysen påpekade även tekniska varningar, exempelvis att en robots.txt-fil saknades eller inte var korrekt konfigurerad. Eftersom webbplatsen är ett lokalt studentprojekt saknar detta praktisk betydelse för indexering, men visar hur verktyg som Lighthouse kan identifiera förbättringsområden inför publicering på en produktionsserver (se Bilaga 5, Bild 2).

Genom att använda Lighthouse under utvecklingsprocessen fick jag en ökad förståelse för hur design- och teknikval, särskilt bildoptimering och layoutstruktur, påverkar webbplatsens prestanda och kvalitet. Testerna bidrog till konkreta förbättringar såsom optimerade bilddimensioner, användning av lazy loading och mer strukturerad resurshantering. Sammantaget har DevTools och Lighthouse varit viktiga verktyg för att utveckla en mer genomtänkt och prestandamedveten webbplats.

9. Utvecklingsprocess

En inledande utmaning i projektet var installationen av Tailwind via npm. Kursmaterialet som användes i undervisningen var delvis baserat på en äldre installationsprocess, vilket gjorde att instruktionerna inte fullt ut stämde överens med den aktuella dokumentationen på Tailwinds officiella webbplats. Detta innebar att jag behövde jämföra olika källor, testa alternativa installationsmetoder och felsöka konfigurationsproblem innan utvecklingsmiljön fungerade korrekt. Erfarenheten gav mig en ökad förståelse för hur snabbt verktyg och ramverk inom webbutveckling förändras samt vikten av att kunna använda aktuell dokumentation och självständigt lösa tekniska problem.

Arbetet med uppgiften innebar också att jag behövde förändra mitt arbetssätt. Tidigare har jag ofta byggt layout genom att först skriva egen CSS och därefter justera detaljer stegvis. Med Tailwind behövde jag i stället tänka mer direkt i struktur, spacing och responsiv logik redan i HTML-koden. Inledningsvis upplevdes detta som ovant och ibland frustrerande, särskilt när jag visste hur jag ville att ett element skulle se ut men ännu inte hade hittat rätt kombination av utility-klasser. Efter hand blev det dock tydligt att detta arbetssätt effektiviserade utvecklingsprocessen. Ju bättre jag lärde känna ramverkets uppbyggnad, desto snabbare gick arbetet.

En viktig lärdom var värdet av återanvändbara designmönster. När jag hade utvecklat fungerande lösningar för exempelvis hero-sektioner, grid-layouter och bild-och-text-block kunde samma struktur användas på flera sidor. Detta bidrog både till en mer effektiv arbetsprocess och till en mer enhetlig visuell identitet på webbplatsen.

Responsiv design blev också en central del av utvecklingsprocessen. Jag upptäckte att det var mer effektivt att arbeta mobile-first och därefter successivt anpassa layouten för större skärmar, vilket ligger i linje med Tailwinds arbetssätt (Tailwind CSS, u.å.-d). De responsiva prefixen gjorde det möjligt att experimentera direkt i HTML, men vissa utmaningar uppstod kring bilders storlek och proportioner i olika skärmstorlekar. Genom att arbeta med wrappers, maxbredder och objektbeskärning kunde jag successivt förbättra den visuella balansen och förstå vikten av proportioner och hierarki i layouten.

10. Summering

Arbetet med detta projekt har tydliggjort hur valet av ramverk påverkar både utvecklingsprocess och slutresultat i webbutveckling. Genom att använda Tailwind CSS som huvudsakligt verktyg för layout och design har jag fått praktisk erfarenhet av hur ett utility-first-baserat arbetssätt kan bidra till mer konsekvent struktur, effektivare utveckling och bättre kontroll över responsivitet. Samtidigt har projektet visat att ett ramverk inte enbart är en teknisk lösning, utan också påverkar hur man planerar, organiserar och prioriterar olika delar av utvecklingsarbetet.

Under projektets gång har jag utvecklat en större förståelse för hur prestanda, tillgänglighet och semantisk struktur samverkar i moderna webbgränssnitt. Arbetet med bildoptimering, validering och responsiv layout har bidragit till att jag blivit mer medveten om hur tekniska beslut påverkar användarupplevelsen. Jag har även fått ökad insikt i vikten av att arbeta systematiskt med återanvändbara komponentmönster och att tidigt planera layout och navigationsstruktur i flersidiga projekt.

Om projektet hade fortsatt finns flera områden jag hade velat utveckla vidare. Till exempel hade desktopnavigationens tillgänglighet kunnat förbättras genom att säkerställa att dropdown-menyer fungerar mer konsekvent med tangentbordsnavigation. Jag hade också sett över användningen av section-element och antingen kompletterat dem med tydliga rubriker eller ersatt vissa med div när de främst används som layoutcontainrar. Detta hade bidragit till en starkare semantisk struktur.

Sammantaget har projektet stärkt både mina tekniska färdigheter och min förmåga att reflektera kring designval och arbetsmetoder. Erfarenheterna har gjort mig mer förberedd på att arbeta med moderna frontendverktyg och att utveckla webbplatser som inte bara fungerar tekniskt, utan också är hållbara, användarvänliga och möjliga att vidareutveckla över tid.

11. Referenser

Bootstrap. (u.å.). *Get started with Bootstrap*. https://getbootstrap.com/docs/5.3/getting-started/introduction/

Bulma. (u.å.). *Documentation*. https://bulma.io/documentation/

Can I Use. (2026). *Can I use… support tables for HTML5, CSS3, etc*. https://caniuse.com/

Chrome Developers. (u.å.). *Lighthouse*. https://developer.chrome.com/docs/lighthouse

Cloudinary. (2025, November 16). *WebP format: Technology, pros, cons, and alternatives*. https://cloudinary.com/guides/front-end-development/webp-format-technology-pros-cons-and-alternatives

Foundation. (u.å.). *Foundation for Sites documentation*. https://get.foundation/sites/docs/

MDN Web Docs. (2025, November 4). *Lazy loading*. https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/Lazy_loading

MDN Web Docs. (2025, November 6). *Responsive images*. https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images

Pollard, B. (2024, August 13). *Browser-level image lazy loading for the web*. web.dev. https://web.dev/articles/browser-level-image-lazy-loading

Pollard, B., & Weiss, Y. (2025, December 3). *Preload responsive images*. web.dev. https://web.dev/articles/preload-responsive-images

Tailwind CSS. (u.å.-a). Documentation. https://tailwindcss.com/

Tailwind CSS. (u.å.-b). *Grid template columns*. https://tailwindcss.com/docs/grid-template-columns

Tailwind CSS. (u.å.-c). *Preflight*. https://tailwindcss.com/docs/preflight

Tailwind CSS. (u.å.-d). *Responsive design*. https://tailwindcss.com/docs/responsive-design

Tailwind CSS. (u.å.-e). Styling with utility classes. https://tailwindcss.com/docs/styling-with-utility-classes

World Wide Web Consortium (W3C). (u.å.). *Markup validation service*. https://validator.w3.org/

12. Bilagor

12.1 Bilaga 1- Produkt-grid

Bild 1. Exempel på produkt-grid som visar hur produkter presenteras i ett responsivt rutnät.

```
<!-- Produkter -->
<section
  id="product"
  class="mb-16 grid grid-cols-1 place-items-center gap-10 md:grid-cols-2 xl:grid-cols-4"
>
  <!-- Produkt -->
  <figure class="flex flex-col items-center text-center">
    <img
      src="image/servis.webp"
      alt="beige tallrik, assiett och skål"
      width="200"
      height="267"
      loading="lazy"
      class="h-auto w-[200px] max-w-full"
    />
    <figcaption class="mt-4">
      <h2 class="text-xl font-medium">X Collection</h2>
      <p class="mt-1">Sett om tallrikar</p>
      <p class="mt-1">599 SEK</p>
```

12.2 Bilaga 2- Egen CSS

Bild 1. Utdrag ur projektets egna CSS-fil.

```
style.css > ...
@import "tailwindcss";

/* Min egna CSS */

/* Typografi */
body {
  font-family: "Roboto", sans-serif;
  color: #121212;
}

h1 {
  line-height: 1.2;
}

h2 {
  line-height: 1.25;
}

h3 {
  line-height: 1.3;
}

p {
  line-height: 1.7;
}

/* Formulär, knappar, textfält */
input,
textarea,
button {
  font: inherit;
}
```

12.3 Bilaga 3- Responsiv bildhantering med picture-element

Bild 1. Kodexempel som visar hur olika hero-bilder laddas beroende på skärmstorlek.

```
<!-- HERO-BILD -->
<picture>
  <source srcset="image/hero.webp" media="(min-width: 1024px)" />
  <source srcset="image/hero.tablet.webp" media="(min-width: 640px)" />
  <img
    src="image/hero.mobil.webp"
    alt="Hösterbjudande"
    width="576"
    height="389"
    class="block w-full h-auto object-cover"
  />
</picture>
```

12.4 Bilaga 4- Validering av HTML

Bild 1. Resultat från W3C HTML Validator som visar identifierade varningar i koden.

Warning **Section lacks heading. Consider using `h2`–`h6` elements to add identifying headings to all sections, or else use a `div` element instead for any cases where no heading is needed.**

From line 400, column 9; to line 402, column 9

```
>↵        <section↵        class="grid grid-cols-1 place-items-center gap-8 pb-20 md:grid-cols-2 xl:grid-cols-3"↵        >↵
```

12.5 Bilaga 5- Lighthouse-analys

Bild 1. Översikt av Lighthouse-resultat (mobiltest)

Bild 2. Identifierade problem inom SEO

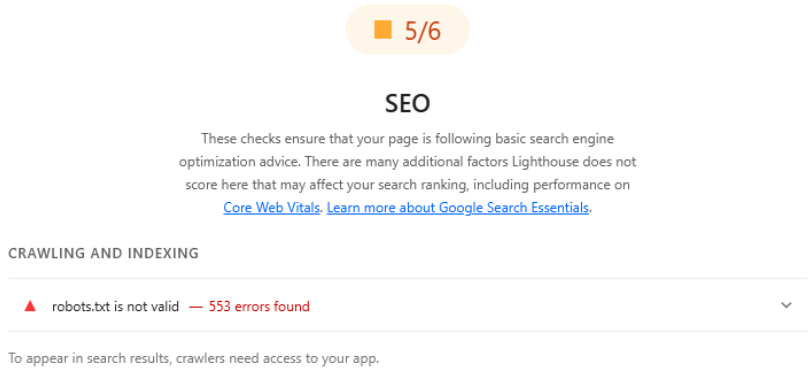
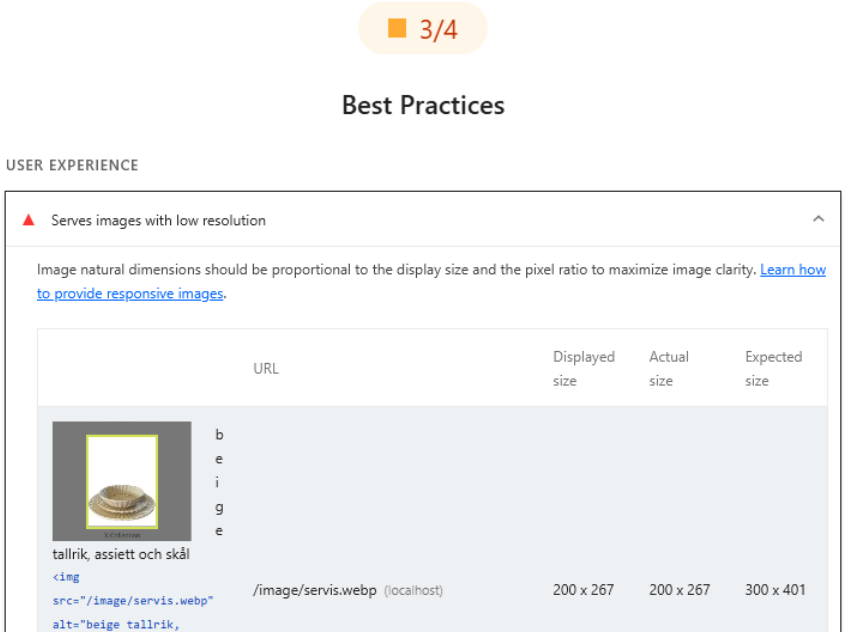
Bild 3. Identifierade problem inom Best Practices